

UNIVERSIDADE FEDERAL DO RIO DE JANEIRO  
CIÊNCIA DA COMPUTAÇÃO  
PROGRAMAÇÃO CONCORRENTE (ICP361)

JOÃO VITOR LOPES ALVARENGA - 120152276  
JORGE VICTOR SIMÃO DE SOUZA - 120130622

TRABALHO FINAL

Comparação de Desempenho entre Métodos Sequencial e Concorrente para Detecção de  
Comunidades em Grafos utilizando o Algoritmo Girvan-Newman

Rio de Janeiro

2024

Sumário:

|                                                        |    |
|--------------------------------------------------------|----|
| 1. Descrição do problema.....                          | 3  |
| 2. Projeto e implementação da solução concorrente..... | 4  |
| 3. Testes de corretude.....                            | 5  |
| 4. Avaliação de desempenho.....                        | 8  |
| 5. Discussão.....                                      | 10 |
| 6. Referências bibliográficas.....                     | 11 |

## 1. Descrição do problema

Comunidades podem ser definidas como um conjunto de elementos que compartilham características em comum. Existem diversos algoritmos dedicados à identificação dessas comunidades.

No estudo de redes sociais, esse é um assunto fundamental, pois uma rede pode conter múltiplas comunidades. Como mencionado, existem diferentes algoritmos para detectar essas comunidades. Neste artigo, focamos no algoritmo Girvan-Newman, que devido à sua complexidade computacional, pode se beneficiar de uma implementação concorrente.

### Método Girvan-Newman

O algoritmo Girvan-Newman funciona removendo as arestas que contêm o maior número de caminhos mínimos passando por elas — essencialmente, removendo as conexões que ligam diferentes partes do grafo. Após essa remoção, as comunidades da rede se tornam evidentes. O método opera identificando a *betweenness centrality* — uma medida da importância de cada aresta ou nó em determinado caminho — e removendo as arestas mais significativas para a rede. Esse processo revela as comunidades naturais do grafo. A execução envolve quatro etapas principais:

- Calcular a Centralidade de Betweenness das Arestas:

Para cada aresta no grafo, calcula-se a centralidade de *betweenness*, que quantifica os caminhos mais curtos que atravessam essa aresta.

- Remover Arestas com Maior Betweenness:

Identifica-se e remove-se as arestas que apresentam maior centralidade de *betweenness* no grafo.

- Detectar Comunidades:

Após a remoção das arestas, o grafo se divide em componentes conectados menores, onde cada componente representa uma comunidade distinta.

- Repetir o Processo:

O processo de cálculo de *betweenness* e remoção de arestas continua iterativamente até atingir o número desejado de comunidades ou até que não seja mais possível remover arestas.

Essas etapas são computacionalmente intensivas, especialmente o cálculo da centralidade de betweenness. Em grafos moderadamente densos, sua complexidade média é  $O(N^3)$ , considerando que o número de arestas geralmente escala como  $O(N^2)$  na prática para grafos com  $N$  vértices. Uma solução concorrente, distribuindo essas tarefas entre múltiplas threads, pode reduzir significativamente o tempo de execução. A utilização de uma solução concorrente pode reduzir significativamente o tempo de execução, distribuindo as tarefas entre múltiplas threads. A paralelização do cálculo de betweenness é particularmente eficiente, pois os cálculos para diferentes vértices são independentes e podem ser executados simultaneamente.

Para verificar a efetividade dessa abordagem, implementamos tanto o método concorrente quanto o sequencial para comparação e análise de desempenho.

## 2. Projeto e implementação da solução concorrente

Para a implementação concorrente tivemos que adaptar alguns trechos do código para aproveitarmos a concorrência. Dentre as principais decisões temos:

- Paralelização por nó:

Descrição: Cada nó do grafo foi tratado como uma unidade independente de cálculo. Isso foi decidido porque o cálculo da intermediação de arestas de um nó não depende diretamente do cálculo de outro nó, tornando o problema adequado para paralelização.

Justificativa: Essa abordagem divide o trabalho uniformemente e reduz o tempo total de processamento ao aproveitar melhor os recursos de hardware.

- Sincronização eficiente com sync.Mutex:

Descrição: Para evitar condições de corrida, utilizou-se um sync.Mutex para proteger a atualização do mapa compartilhado que armazena os valores de intermediação das arestas.

Justificativa: Embora o uso do mutex introduza alguma sobrecarga, ele garante a consistência dos dados, que é essencial para a correta execução do algoritmo.

- Redução do escopo de bloqueios:

Descrição: O bloqueio do mutex foi limitado ao menor escopo possível, ou seja, apenas durante a atualização do mapa compartilhado.

Justificativa: Isso minimiza o impacto da sincronização e maximiza a paralelização.

- Estratégia de agregação local:

Descrição: Cada goroutine calcula a intermediação localmente em um mapa separado antes de consolidar os resultados no mapa global.

Justificativa: Isso reduz a frequência de bloqueios do mutex, melhorando o desempenho.

### 3. Testes de corretude

Para a testagem do programa utilizamos 13 variações de grafos, Dentre eles:

1. Grafo cíclico com conexão pequeno

Descrição: Um grafo onde os nós formam um ciclo fechado, e cada nó está conectado apenas aos seus vizinhos imediato anterior e seguinte.

Justificativa: Testar a corretude e desempenho dos algoritmos sequencial e concorrente com grafos simples onde há caminhos mínimos entre qualquer nó.

2. Grafo desconexo pequeno

Descrição: Um grafo com múltiplos componentes desconexos, ou seja, conjuntos de nós que não possuem conexões entre si.

Justificativa: Testar a corretude e desempenho ao identificar corretamente componentes desconexos como comunidades distintas.

3. Grafo linear pequeno

Descrição: Um grafo em forma de cadeia onde cada nó está conectado apenas ao nó anterior e ao próximo, sem formar ciclos.

Justificativa: Testar a corretude para calcular caminhos mínimos em estruturas lineares e avaliar o desempenho em grafos simples com poucos caminhos redundantes.

4. Grafo completo pequeno

Descrição: Um grafo onde todos os nós estão conectados a todos os outros.

Justificativa: Testar a corretude em cenários de máxima densidade de conexões e medir o impacto no desempenho devido ao grande número de caminhos redundantes.

5. Grafo raro pequeno

Descrição: Um grafo esparsos com muito poucas arestas em relação ao número de nós. Muitos nós podem estar isolados ou quase isolados.

Justificativa: Testar a corretude em cenários de baixa conectividade e verificar como o algoritmo lida com nós isolados ou quase isolados. Também avalia o desempenho em grafos muito esparsos.

6. Grafo denso pequeno

Descrição: Um grafo com alta densidade de conexões, mas que não é completamente conectado.

Justificativa: Testar a corretude em grafos com muitos caminhos redundantes e medir o impacto do aumento da densidade nas implementações sequenciais e concorrentes.

7. Grafo pequeno com comunidades

Descrição: Um grafo pequeno com dois ou mais subgrupos densamente conectados, mas com poucas conexões entre os subgrupos.

Justificativa: Testar a capacidade do algoritmo em identificar corretamente as comunidades e medir o desempenho em grafos com estrutura hierárquica.

8. Grafo escalado médio

Descrição: Um grafo com um número moderado de nós e arestas, escalado para representar cenários intermediários.

Justificativa: Testar a corretude e medir a escalabilidade em cenários mais próximos de aplicações práticas.

9. Grafo escalado grande

Descrição: Um grafo com muitos nós e arestas, próximo ao limite prático de execução.

Justificativa: Avaliar a eficiência e escalabilidade do algoritmo, especialmente na versão concorrente, em cenários que desafiem o desempenho

#### 10. Grafo completo grande

Descrição: Um grafo grande onde todos os nós estão conectados a todos os outros.

Justificativa: Testar a corretude em grafos densos de grande porte e medir o impacto no desempenho devido à alta densidade de conexões.

#### 11. Grafo grande com comunidades

Descrição: Um grafo grande com várias comunidades densamente conectadas, mas com poucas conexões entre as comunidades.

Justificativa: Verificar a capacidade do algoritmo em detectar comunidades em grafos grandes e medir a eficiência em cenários próximos de aplicações reais.

#### 12. Grafo facebook

Descrição: Representação de uma rede social com nós densamente conectados dentro de comunidades e conexões esparsas entre comunidades.

Justificativa: Testar a corretude e o desempenho do algoritmo em grafos realistas, representando padrões típicos de redes sociais.

#### 13. Grafo emails

Descrição: Representação de uma rede de trocas de e-mails, com padrões hierárquicos e comunidades baseadas em interações.

Justificativa: Avaliar a precisão do algoritmo em identificar clusters reais e medir a eficiência em grafos com características do mundo real.

Os resultados dos testes de desempenho mostram diferenças significativas entre as implementações sequencial e concorrente, dependendo do tamanho e complexidade dos grafos. Em grafos pequenos, a versão concorrente não apresentou ganhos expressivos e, em alguns casos, teve desempenho ligeiramente inferior ao método sequencial. Isso ocorre porque, em grafos com poucos nós e arestas, o overhead gerado pela criação e sincronização de múltiplas threads supera os benefícios da paralelização. Nessas situações, as operações realizadas em paralelo, como o cálculo de intermediação para poucos nós, não são suficientes para compensar o custo adicional associado ao gerenciamento de threads.

Por outro lado, em grafos grandes e densos, a versão concorrente demonstrou desempenho significativamente superior. O motivo principal é que, nesses cenários, a paralelização permite dividir a carga computacional intensiva entre múltiplas threads, reduzindo o tempo necessário para calcular a intermediação das arestas e outras etapas críticas do algoritmo. À medida que o número de nós e arestas aumenta, o trabalho distribuído entre as threads se torna proporcionalmente mais eficiente, pois o impacto do overhead inicial se dilui diante do volume de operações paralelas.

Em grafos complexos e realistas, como os grafos Facebook e de e-mails, a abordagem concorrente mostrou-se particularmente eficiente, lidando com a alta conectividade e densidade de forma escalável, confirmando sua superioridade em aplicações práticas com grafos de grande porte.

#### 4. Avaliação de desempenho

Para avaliar o desempenho do programa, utilizamos o seguinte hardware:

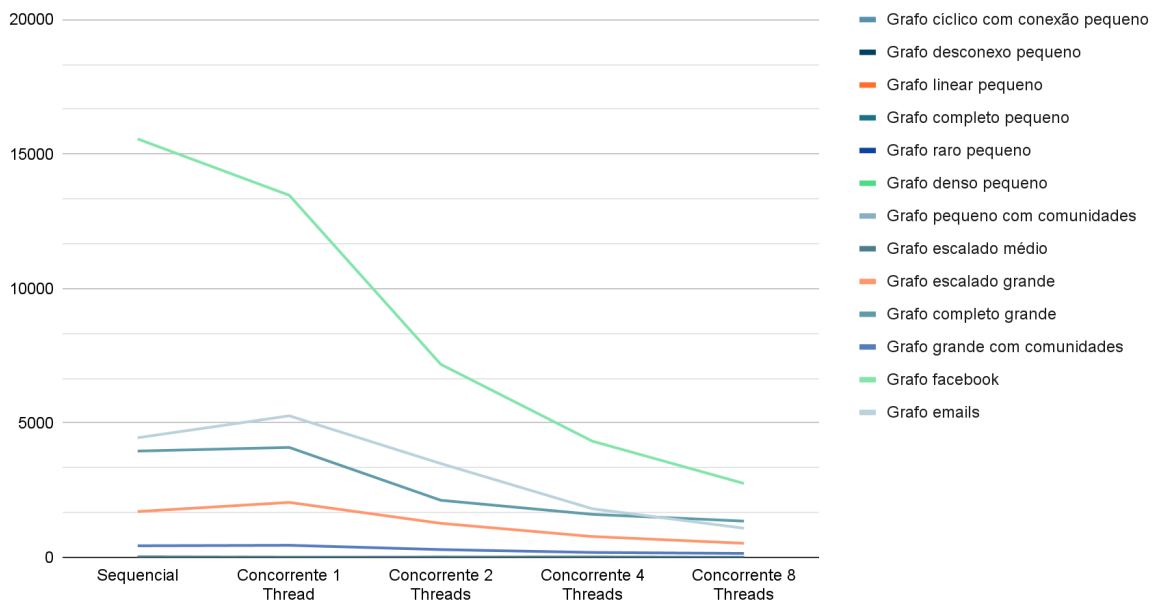
- Processador: 12th Gen Intel(R) Core(TM) i7-1255U @ 1.70 GHz
- Número de Núcleos: 10
- Número de Threads: 12
- Memória RAM: 16.0 GB (15.7 GB utilizável)

O programa foi executado 5 vezes para cada número de threads, garantindo maior consistência nos resultados. Em cada execução, foi medido o tempo de execução, que foi então convertido para milissegundos. O valor final utilizado para análise foi a média aritmética dos tempos obtidos. Essa abordagem ajudou a minimizar o impacto de flutuações ocasionais no desempenho, como processos do sistema ou variações momentâneas de carga.



Com os tempos médios calculados, foi possível comparar os tempos médios da implementação sequencial e da concorrente, com diferentes números de threads, e determinar se houve ganhos de desempenho com a utilização de processamento paralelo

**Tempo Médio de Execução em ms**

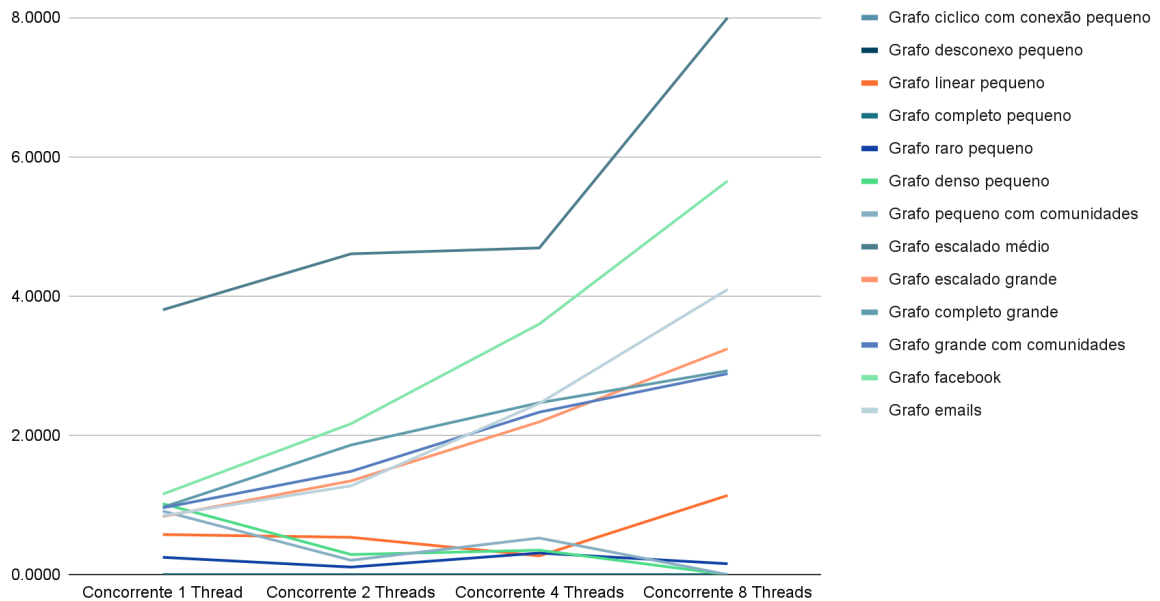


Ao avaliar o desempenho dos algoritmos sequencial e concorrente para diferentes tipos de grafos, observa-se que o tempo de execução varia significativamente dependendo da estrutura do grafo e do número de threads utilizadas. Em grafos pequenos, o desempenho concorrente com várias threads frequentemente apresenta tempos superiores ao sequencial, indicando que a sobrecarga de gerenciamento das threads não compensa para tamanhos reduzidos. No entanto, para grafos escalados ou grandes, como "Grafo escalado grande" e "Grafo completo grande", o desempenho melhora significativamente com o aumento do número de threads, especialmente até 4 threads, onde há uma redução notável no tempo de execução em comparação ao sequencial.

Além do tempo médio de execução, também foi calculada a aceleração para avaliar o desempenho ao variar a quantidade de threads. Para o cálculo da aceleração, utilizamos:

$$Aceleração = \frac{Tempo\ Sequencial}{Tempo\ Paralelo}$$

## Aceleração

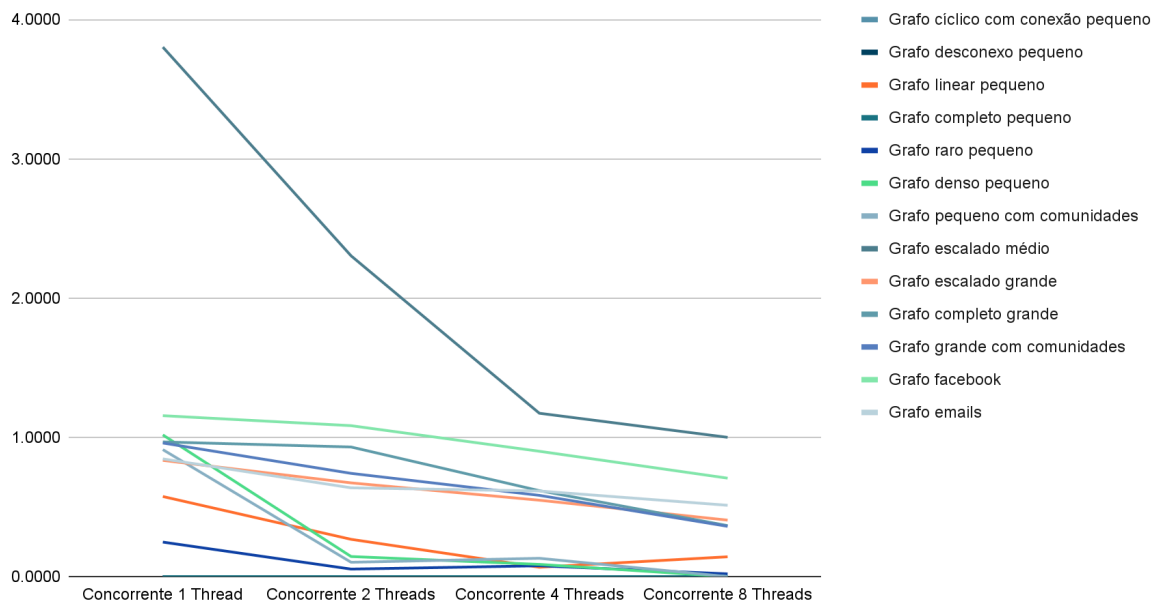


Ao analisar o gráfico de aceleração, que mede a relação entre o tempo de execução do algoritmo sequencial e o concorrente, observamos variações significativas dependendo do tipo de grafo e do número de threads utilizadas. Em grafos pequenos, a aceleração é baixa ou praticamente inexistente, indicando que o uso de threads não oferece vantagem nesses casos devido à sobrecarga introduzida. Em grafos maiores, como o "Grafo escalado médio," "Grafo escalado grande," e "Grafo completo grande," a aceleração aumenta com o número de threads, alcançando valores acima de 7.9x para 8 threads no "Grafo escalado médio." No entanto, a eficiência não cresce linearmente com o aumento de threads, e em alguns casos, como no "Grafo raro pequeno" e "Grafo denso pequeno," observa-se aceleração reduzida com mais threads, possivelmente devido a limitações de paralelismo ou contenção de recursos. O resultado evidencia que o desempenho ideal depende do equilíbrio entre o custo de gerenciamento das threads e a complexidade computacional do grafo processado.

E, para finalizar, calculamos a eficiência, dada por:

$$Eficiência = \frac{Aceleração}{Número\ Threads}$$

## Eficiência



Ao avaliar a eficiência, definida como a aceleração dividida pelo número de threads, observamos que o desempenho do algoritmo concorrente varia significativamente conforme o tipo de grafo e o número de threads. Para grafos pequenos, como o "Grafo cíclico com conexão pequeno" e "Grafo desconexo pequeno," a eficiência é nula, indicando que o uso de threads não proporciona ganho algum. Em grafos maiores, como "Grafo escalado médio" e "Grafo escalado grande," a eficiência é mais alta com menos threads (cerca de 3.80 para 1 thread e 2.30 para 2 threads no "Grafo escalado médio"), mas diminui gradativamente com o aumento de threads, chegando a menos de 1.0 para 8 threads. Esse padrão reflete a sobrecarga crescente de comunicação e sincronização conforme mais threads são utilizadas. Para grafos altamente conectados ou densos, a eficiência é especialmente baixa com múltiplas threads, como visto no "Grafo raro pequeno" e "Grafo completo grande," onde o custo de paralelismo supera os benefícios computacionais. Esses resultados evidenciam a importância de otimizar o número de threads com base no tamanho e complexidade do grafo para alcançar eficiência máxima.

## 5. Discussão

Em geral, os resultados foram conforme o esperado. Como vimos nas primeiras aulas, a computação concorrente se beneficia de cálculos intensivos, como na multiplicação de matrizes, especialmente em matrizes grandes. Já em matrizes pequenas, o ganho era bem pequeno, ou até mesmo a versão sequencial se saía melhor. Embora a implementação não utilize matrizes, mas sim hashmaps para representar os grafos, os resultados obtidos com o algoritmo de Girvan-Newman foram similares ao que se observa em aplicações com alto volume de cálculos intensivos, como na multiplicação de matrizes.

Nos testes, verificamos que em grafos pequenos, o overhead da criação e sincronização de threads superava os benefícios da concorrência, fazendo com que a versão sequencial tivesse um desempenho superior, como era o esperado. No entanto, em grafos maiores, o cenário foi diferente: havia mais trabalho a ser distribuído entre as threads, o que maximizou o aproveitamento dos múltiplos núcleos disponíveis no hardware, como era o esperado.

Até o momento, não pensamos em fazer melhorias. Conseguimos chegar a um resultado esperado, mas com o tempo acreditamos que novas ideias para o código surgirão. Gostaríamos de expandir o código para resolver mais casos reais, como nos últimos grafos que apresentamos.

Sobre o trabalho, de início enfrentamos dificuldades. Começamos a implementar em C, mas percebemos que havia muitas limitações, o que fazia com que o desempenho da versão sequencial fosse sempre melhor. Dessa forma, decidimos optar pelo Go, que nos dava mais liberdade e ferramentas para o paralelismo que possibilitaram um resultado desejável.

## 6. Referências bibliográficas

<https://medium.com/@thamireslopes/an%C3%A1lise-de-m%C3%A9todos-de-detec%C3%A7%C3%A3o-de-comunidades-em-redes-complexas-7767b1b31ab7>  
<https://snap.stanford.edu/data/>  
<https://snap.stanford.edu/data/email-Eu-core.html>  
[https://en.wikipedia.org/wiki/Girvan%E2%80%93Newman\\_algorithm](https://en.wikipedia.org/wiki/Girvan%E2%80%93Newman_algorithm)  
<https://youtu.be/JcBVE9OKATk?si=u2boj22b6uiY0ZGN>  
<https://youtu.be/F4RVBAGJcFY?si=EBjBv-OAvUi0UihU>  
<https://oaji.net/articles/2022/3603-1666575696.pdf>  
<https://github.com/jorgevictor/Trabalho-Final-Concorrente/tree/main>

